

UNIT TESTING REPORT- Healthcare Management System

OVERALL TEST SUMMARY

Total Tests	Passed	Failed	Skipped	Build Status
40	40	0	0	BUILD SUCCESS

SECTION 1: APPOINTMENT SERVICE TESTS (AppointmentServiceTest - 13 Tests)

#	Test Function Name	Test Scenario / Input	Expected Result	Status
1	testCreateAppointment_Success	Valid booking (Doctor ID, Patient ID, Future Date, Time)	Appointment saved with status 'Booked' and consultation fee	PASS
2	testCreateAppointment_PatientNotFound_ThrowsException	Booking with non-existent Patient ID: 999	Fails: Throws error 'Patient not found'	PASS
3	testCreateAppointment_DoctorNotFound_ThrowsException	Booking with non-existent Doctor ID: 999	Fails: Throws error 'Doctor not found'	PASS
4	testCreateAppointment_MissingDate_ThrowsException	Booking with missing date (null)	Fails: Throws error 'Appointment date is required.'	PASS
5	testCreateAppointment_PastDate_ThrowsException	Booking with past date (e.g. '2020-01-01')	Fails: Throws error 'Cannot book an appointment for a past date'	PASS
6	testCreateAppointment_MissingTime_ThrowsException	Booking with missing time slot	Fails: Throws error 'Appointment time is required.'	PASS
7	testCreateAppointment_DoctorDoubleBooking_ThrowsException	Doctor already booked for same date and time slot	Fails: Throws error 'Doctor is already booked at this time.'	PASS
8	testCreateAppointment_PatientDoubleBooking_ThrowsException	Patient already booked with another doctor at same time	Fails: Throws error 'Patient already has an appointment at this time.'	PASS
9	testCancelAppointment_Success	Cancel existing appointment by ID: 501	Appointment status updated to 'CANCELLED'	PASS
10	testRescheduleAppointment_Success	Reschedule to valid new future date and time slot	Appointment updated with new date/time, status remains 'Booked'	PASS
11	testRescheduleAppointment_PastDate_ThrowsException	Reschedule to past date (e.g. '2020-01-01')	Fails: Throws error 'Cannot reschedule appointment to a past date'	PASS
12	testUpdatePrescription_Success	Doctor writes and updates prescription notes	Prescription saved successfully on appointment	PASS
13	testUpdatePrescription_EmptyPrescription_ThrowsException	Doctor submits blank / empty prescription (e.g. ' ')	Fails: Throws error 'Prescription note cannot be empty.'	PASS

SECTION 2: USER SERVICE & VALIDATION TESTS (UserServiceTest - 15 Tests)

#	Test Function Name	Test Scenario / Input	Expected Result	Status
1	testRegisterPatient_Success	Register new patient with valid details	User saved with PATIENT role and patient record initialized	PASS
2	testRegisterDoctor_WithShiftAndHours	Register new doctor with shift timing and working hours	User saved with DOCTOR role and doctor record initialized	PASS
3	testRegisterDoctor_AppliesDefaultsWhenFieldsBlank	Register doctor with blank specialization and qualification	Applies default fallbacks ('General Physician', 'MBBS, MD')	PASS
4	testRegisterUser_DefaultRolePatientWhenNull	Register user with role as null	Defaults user role to PATIENT	PASS
5	testRegisterUser_InvalidName_ThrowsException	Name containing numbers (e.g. '234')	Fails: Throws error 'Invalid name. Name must contain only letters'	PASS
6	testRegisterUser_InvalidPhone_ThrowsException	Phone containing text (e.g. 'sudhansh')	Fails: Throws error 'Invalid phone number. Must be a 10-digit number'	PASS
7	testRegisterUser_InvalidEmail_ThrowsException	Malformed email format (e.g. 'not-an-email')	Fails: Throws error 'Invalid email format'	PASS
8	testRegisterUser_ShortPassword_ThrowsException	Password shorter than 6 characters (e.g. '123')	Fails: Throws error 'Password must be at least 6 characters long'	PASS
9	testRegisterUser_DuplicateEmail_ThrowsException	Registering with an already registered email in DB	Fails: Throws error 'Email is already registered'	PASS
10	testSaveUser_PatientRole_CreatesPatientIfNotExists	Save user with PATIENT role (creates profile if missing)	Creates linked patient profile in database	PASS
11	testSaveUser_DoctorRole_CreatesDoctorIfNotExists	Save user with DOCTOR role (creates profile if missing)	Creates linked doctor profile with default fee/hours	PASS
12	testLogin_Success	Login with valid registered email and password	Returns matching authenticated user details	PASS
13	testLogin_InvalidPassword_ThrowsException	Login with incorrect password	Fails: Throws error 'Invalid password'	PASS
14	testLogin_UserNotFound_ThrowsException	Login with unregistered email	Fails: Throws error 'User not found'	PASS
15	testGetAllUsers_ReturnsUserList	Fetch all users list for admin	Returns list of all registered users	PASS

SECTION 3: DOCTOR SERVICE TESTS (DoctorServiceTest - 8 Tests)

#	Test Function Name	Test Scenario / Input	Expected Result	Status
1	testSaveDoctor_Success	Save and create doctor entity	Doctor profile saved with specialization and consultation fee	PASS
2	testGetAllDoctors_ReturnsList	Get list of all registered doctors	Returns list containing all doctor profiles	PASS
3	testUpdateAvailability_Success	Update doctor availability status to 'Busy'	Availability updated to 'Busy'	PASS
4	testUpdateAvailability_DoctorNotFound_ThrowsException	Update availability for non-existent doctor ID: 999	Fails: Throws error 'Doctor not found'	PASS
5	testUpdateConsultationFee_Success	Update consultation fee to ₹2000	Consultation fee updated to ₹2000	PASS
6	testUpdateConsultationFee_DoctorNotFound_ThrowsException	Update consultation fee for non-existent doctor ID: 999	Fails: Throws error 'Doctor not found'	PASS
7	testUpdateShiftAndWorkingHours_Success	Update shift to 'Night' and	Shift and daily working hours	PASS

		working hours to 6	updated	
8	testUpdateShift_DoctorNotFound_ThrowsException	Update shift for non-existent doctor ID: 999	Fails: Throws error 'Doctor not found'	PASS

SECTION 4: PATIENT SERVICE TESTS (PatientServiceTest - 4 Tests)

#	Test Function Name	Test Scenario / Input	Expected Result	Status
1	testSavePatient_Success	Save and create patient entity	Patient profile saved with gender and blood group	PASS
2	testGetAllPatients_ReturnsList	Get list of all registered patients	Returns list containing all patient records	PASS
3	testUpdateMedicalHistory_Success	Update patient medical history	Medical history updated successfully	PASS
4	testUpdateMedicalHistory_PatientNotFound_ThrowsException	Update medical history for non-existent patient ID: 999	Fails: Throws error 'Patient not found'	PASS

1. Terminal Test Output

```
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.056 s -- in com.healthcare.healthcare_backend.service.AppointmentServiceTest
[INFO] Running com.healthcare.healthcare_backend.service.DoctorServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.104 s -- in com.healthcare.healthcare_backend.service.DoctorServiceTest
[INFO] Running com.healthcare.healthcare_backend.service.PatientServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.054 s -- in com.healthcare.healthcare_backend.service.PatientServiceTest
[INFO] Running com.healthcare.healthcare_backend.service.UserServiceTest
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.241 s -- in com.healthcare.healthcare_backend.service.UserServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 40, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.376 s
[INFO] Finished at: 2026-09-09T23:18:16+05:30
[INFO] -----
PS C:\Users\sudha\Desktop\healthcare\backend\healthcare\healthcare-backend>
```

2. User Registration DTO (This is to check the constraints at input)

```
public class RegisterRequest { 23 usages 1 Sudhansh *

    @NotBlank(message = "Name is required") 2 usages
    @Pattern(regexp = "^[a-zA-Z\\s]{2,50}$", message = "Name must contain only letters and spaces (2 to 50 characters)")
    private String name;

    @NotBlank(message = "Email is required") 2 usages
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "Password is required") 2 usages
    @Size(min = 6, message = "Password must be at least 6 characters")
    private String password;

    @NotBlank(message = "Phone number is required") 2 usages
    @Pattern(regexp = "[0-9]{10}$", message = "Phone must be a valid 10-digit number")
    private String phone;

    private Role role; 2 usages
}
```

3.Appointment Request DTO (This is for appointment)

```
public class AppointmentRequest { 21 usages 1 Sudhansh *

    @NotNull(message = "Patient ID is required") 2 usages
    private Long patientId;

    @NotNull(message = "Doctor ID is required") 2 usages
    private Long doctorId;

    @NotBlank(message = "Appointment date is required") 2 usages
    private String appointmentDate;

    @NotBlank(message = "Appointment time is required") 2 usages
    private String appointmentTime;
```

4. Global Exception Handler (This is centralized handler file for various exception)

```
3 > import ...
4 @RestControllerAdvice no usages
5 public class GlobalExceptionHandler {
6
7     @ExceptionHandler(MethodArgumentNotValidException.class) no usages
8     public ResponseEntity<ErrorResponse> handleValidationExceptions(MethodArgumentNotValidException ex) {
9         String errorMessage = ex.getBindingResult().getFieldErrors().stream().map(FieldError::getDefaultMessage).stream().collect(Collectors.joining(", "));
10
11         ErrorResponse errorResponse = new ErrorResponse(
12             HttpStatus.BAD_REQUEST.value(),
13             "Validation Failed",
14             errorMessage
15         );
16         return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
17     }
18
19     @ExceptionHandler(IllegalArgumentException.class) no usages
20     public ResponseEntity<ErrorResponse> handleIllegalArgumentException(IllegalArgumentException ex) {
21         ErrorResponse errorResponse = new ErrorResponse(
22             HttpStatus.BAD_REQUEST.value(),
23             "Invalid Argument",
24             ex.getMessage()
25         );
26         return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
27     }
28
29     public class GlobalExceptionHandler {
30         public ResponseEntity<ErrorResponse> handleIllegalArgumentException(IllegalArgumentException ex) {
31         }
32
33         @ExceptionHandler(AppointmentAlreadyExistsException.class) no usages
34         public ResponseEntity<ErrorResponse> handleAppointmentAlreadyExistsException(AppointmentAlreadyExistsException ex) {
35             ErrorResponse errorResponse = new ErrorResponse(
36                 HttpStatus.CONFLICT.value(),
37                 "Conflict",
38                 ex.getMessage()
39             );
40             return new ResponseEntity<>(errorResponse, HttpStatus.CONFLICT);
41         }
42
43         @ExceptionHandler(RuntimeException.class) no usages
44         public ResponseEntity<ErrorResponse> handleRuntimeException(RuntimeException ex) {
45             ErrorResponse errorResponse = new ErrorResponse(
46                 HttpStatus.BAD_REQUEST.value(),
47                 "Bad Request",
48                 ex.getMessage()
49             );
50             return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
51         }
52
53         @ExceptionHandler(Exception.class) no usages
54         public ResponseEntity<ErrorResponse> handleGenericException(Exception ex) {
55             ErrorResponse errorResponse = new ErrorResponse(
56                 HttpStatus.INTERNAL_SERVER_ERROR.value(),
57                 "Internal Server Error",
58                 ex.getMessage() != null ? ex.getMessage() : "An unexpected error occurred"
59             );
60             return new ResponseEntity<>(errorResponse, HttpStatus.INTERNAL_SERVER_ERROR);
61         }
62     }
63 }
```